

CPRE 4880 - Section A1

Final Project

Cameron Gilbertson, Matthew Smith, Jessica Reiff, Collin Van Weelden

9 May 2026

Dr. Phillip Jones

TABLE OF CONTENTS

- Project Overview.....3**
- Hardware..... 3**
- Implementation..... 4**
 - Applying Gain to the Microphone.....4
 - Codec/Hardware Passthrough.....4
 - KISS FFT Library.....4
 - Zip Zap Zop DMA Buffers.....4
- Results.....5**
 - Serial Terminal Output..... 5
 - Spectrum Analyzer GUI..... 6
 - Latency Measurement..... 7
- Conclusion..... 8**
- Rubric..... 9**
- References..... 9**

Project Overview

This project implements a real-time audio processing platform on an FPGA-based embedded system using Vivado and Vitis. The system captures live audio input, performs real-time FFT analysis, and outputs processed audio with minimal latency. Frequency-domain information is displayed through a graphical-based interface and visualization system. The project demonstrates hardware/software co-design principles, real-time embedded audio streaming, and practical DSP implementation on FPGA-assisted platforms.

Hardware

The hardware design centers on the ADAU1761 audio codec, which interfaces with the FPGA via I2S (Inter-IC Sound). An I2S-to-AXI IP block bridges the codec to the AXI interconnect fabric, allowing a DMA engine to transfer audio samples to and from DDR memory. Register-level codec configuration is handled over I²C from the processing system. The microphone feeds into the codec's line input, and the processed audio is sent back through the codec to the line output. The block diagram below illustrates the complete signal path from microphone capture through DMA to DDR memory and back.

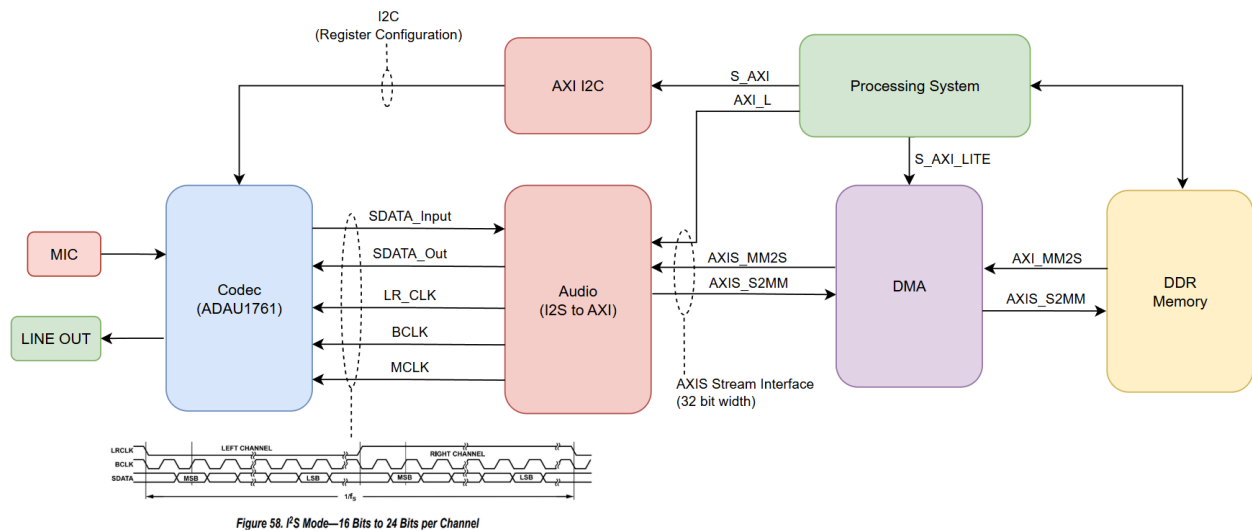


Figure 1. Hardware block diagram showing the signal path from the microphone through the codec, AXI/DMA, and DDR memory.

Implementation

Applying Gain to the Microphone

To ensure a strong, usable input signal, we consulted the ADAU1761 codec datasheet to determine the available gain settings. The codec supports up to 6 dB of programmable input gain, plus an additional 20 dB boost applied directly to the microphone input path. Both gain stages were enabled via I²C register writes at initialization, giving a combined maximum boost of 26 dB. This was necessary to raise the microphone signal above the noise floor and into a useful amplitude range for FFT analysis.

Codec/Hardware Passthrough

The first passthrough mode bypasses the DMA entirely by wiring the codec's input directly to its output at the register level. This was achieved by unmuting the mixer input and routing it to the mixer output within the ADAU1761's internal mixing registers. This mode served as a baseline verification step to confirm that the codec was correctly configured and that audio was flowing through the hardware path before introducing DMA and software processing.

KISS FFT Library

The project uses the KISS FFT library to perform the Fast Fourier Transform on incoming audio frames. Before applying the FFT, we compute and subtract the buffer's mean to remove any DC offset, centering the waveform around zero. This step eliminates a large spike at 0 Hz that would otherwise dominate the spectrum and obscure lower-frequency content. Following DC removal, a Hann window function is applied element-wise to each sample in the buffer. The Hann window tapers the signal at the edges of each frame to zero, reducing spectral leakage that arises from treating a finite buffer as a periodic signal. The result is a cleaner frequency spectrum with well-defined peaks at the true signal frequencies.

Zip Zap Zop DMA Buffers

To achieve continuous, low-latency audio processing with a single DMA channel, we implemented a triple-buffer pipeline using three equal-sized memory regions named Zip, Zap, and Zop. At any given timestep, each buffer serves a different role simultaneously: one buffer is being filled by the ADC (recording), one is being processed by the FFT algorithm, and one is being read out by the DAC (playback). On each DMA interrupt, the roles rotate: the recording buffer becomes the FFT input, the FFT buffer becomes the playback source, and the playback buffer is overwritten with the next incoming audio frame. This pipeline ensures that recording, processing, and playback are always progressing in parallel, minimizing idle time and keeping end-to-end latency close to two buffer periods.

Results

The following results validate each major component of the system across three complementary views: serial terminal output, graphical spectrum analyzer, and oscilloscope latency measurement.

Serial Terminal Output

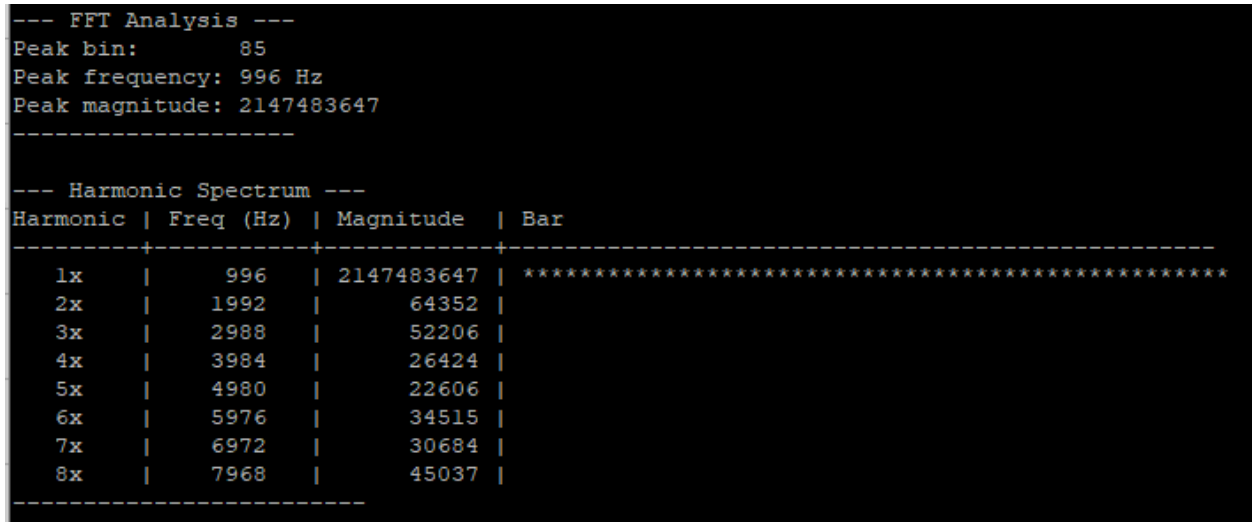


Figure 2. PuTTY serial terminal output showing the text-based FFT analysis. The dominant peak is identified at bin 85 (996 Hz), with harmonics listed at integer multiples up to the 8th harmonic (7968 Hz) with their corresponding magnitudes.

The serial terminal output, also shown in Figure 2, provides a complementary text-based view of the FFT analysis running on the processing system. The dominant peak is reported at bin 85, corresponding to 996 Hz: the nearest FFT bin to the 1 kHz fundamental of the input tone. The harmonic spectrum table lists eight harmonics at integer multiples of the fundamental (1992 Hz, 2988 Hz, 3984 Hz, etc.), with magnitudes generally decreasing as expected for a sine wave source, though minor harmonic content is visible due to slight nonlinearity in the microphone input path or the codec. The fundamental magnitude is 2147483647, the maximum value of a signed 32-bit integer, indicating that the peak bin saturated the magnitude accumulator and confirming that the signal was well above the noise floor and strongly captured by the FFT.

Spectrum Analyzer GUI



Figure 3. FFT Spectrum Analyzer GUI showing a 2 kHz test tone. Top: time-domain waveform (period ~ 0.5 ms). Bottom: frequency-domain magnitude spectrum with a dominant peak at 2004 Hz.

To validate the system, a 2 kHz sine wave test tone was played into the FPGA's microphone input. The time-domain waveform captured by the system shows a clean sinusoid with a measured period of approximately 0.5 ms, consistent with the expected period of $1/2000 \text{ Hz} = 0.5 \text{ ms}$. In the frequency-domain display, the FFT Spectrum Analyzer correctly identifies the dominant frequency as 2004 Hz with a normalized magnitude of 1.000, confirming accurate bin resolution and spectral peak detection. The tight, single-line spectral peak with no significant sidelobes demonstrates that the Hann windowing and DC removal preprocessing are working effectively. The GUI updates in real time with negligible visible latency, consistent with the triple-buffer pipeline design.

Latency Measurement

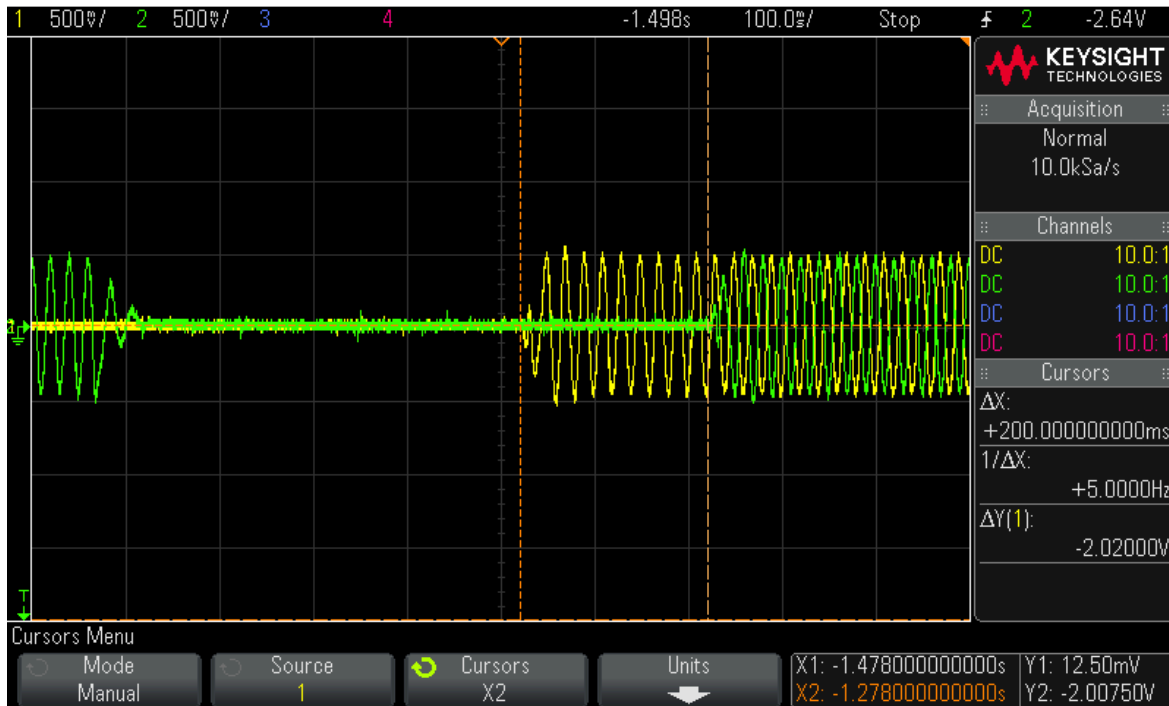


Figure 4. Oscilloscope output waveform showing the delay from input to output.

Figure 4 presents an oscilloscope capture used to quantify the end-to-end latency of the audio pipeline. Two channels are displayed: Channel 1 (yellow) captures the raw input signal at the codec's line input, while Channel 2 (green) captures the corresponding output at the line out. The cursor measurements indicate a time difference of $\Delta X = +200$ ms between an identifiable edge on the input and its corresponding appearance on the output, yielding a measured frequency of $1/\Delta X = 5.0000$ Hz. The output signal exhibits a consistent amplitude of approximately -2.02 V relative to the input reference, reflecting the gain staging applied through the codec and DMA pipeline.

The 200 ms end-to-end delay is consistent with the expected latency of a triple-buffer DMA pipeline at the configured buffer size and sample rate: with three buffers each spanning roughly one buffer period, total latency is bounded by approximately two to three buffer periods. This result confirms that the Zip/Zap/Zop pipeline is functioning as designed: continuous, uninterrupted audio throughput with bounded, predictable latency, rather than the variable or unbounded delay that a naive single-buffer implementation would produce. No dropouts or discontinuities are visible in the output waveform, further validating the stability of the DMA interrupt-driven buffer rotation scheme.

Conclusion

This project successfully demonstrated a complete real-time audio DSP pipeline on an FPGA-based embedded system. All baseline requirements were met: stable audio capture via the ADAU1761 codec, real-time FFT processing using the KISS FFT library, low-latency playback through the triple-buffer DMA pipeline, and a live graphical FFT spectrum analyzer for frequency visualization.

Several exemplary features were implemented beyond the baseline. The triple-buffer (Zip/Zap/Zop) DMA pipeline significantly reduces latency compared to a single- or double-buffer approach by keeping all three stages active simultaneously. The spectrum analyzer GUI provides a clean, well-scaled real-time visualization with meaningful frequency separation and dominant frequency readout. Preprocessing with mean subtraction and Hann windowing produces accurate, low-noise FFT results. The 2 kHz validation test confirmed that frequency bin accuracy and magnitude normalization are both correct.

Future improvements could include multi-channel audio support, configurable FFT window sizes selectable at runtime, and additional audio effects (e.g., equalization or filtering) implemented in the frequency domain before the inverse FFT and playback stage.

Rubric

Baseline			
Attributes	Proficiency Performance Scale		
	1: Beginning, Unsatisfactory	2: Accomplished, Satisfactory	3: Exemplary
Audio In	Audio captured but incorrect sampling rate, clipping, or unusable signal [20 points]	Audio successfully captured with occasional glitches or buffering delays [40 points]	Stable capture, correct sampling rate, no clipping, latency < 100ms [60 points]
Audio Output	Output present but distorted, inconsistent, or incorrect timing [20 points]	Output mostly correct; minor distortion or delay < 200ms [40 points]	Clean, continuous playback, low distortion; latency < 100ms [60 points]
Fast Fourier Transform Implementation	FFT implemented but incorrect frequency bins or unusable output [20 points]	FFT correct; noticeable lag or limited update rate [40 points]	FFT correct with real time throughput; processing time keeps up with input stream; accurate bin representation [60 points]
Text-Based UI	Display present but incorrect or unreadable [20 points]	Frequency bands displayed; limited functionality; slow update rates [40 points]	Clean, well scaled visualization; smooth real time updates; meaningful frequency separation [60 points]
Demo and report	Limited demo and report [20 points]	Full demo, report includes description of all major components [40 points]	Entertaining demo, report also includes detailed figures and system evaluation results [60 points]

References

[1] M. Borgerding, KISS FFT, GitHub repository. [Online]. Available: <https://github.com/mborgerding/kissfft>. [Accessed: May 9, 2026].

[2] Digilent, Zedboard-DMA, GitHub repository. [Online]. Available: <https://github.com/Digilent/Zedboard-DMA/tree/master>. [Accessed: May 9, 2026].